

Secrets and masking: Secrets Manager, Parameter Store and redaction at source

Closing Domain 5. Skill 5.3.4 – masking and anonymisation – is new in SCS-C03.

A secret that has been logged is not a secret. Neither is one that never rotates.

Secrets Manager, or Parameter Store?

SECRETS MANAGER

Built for credentials

- Automatic rotation, including managed rotation
- Resource policies for cross-account access
- Cross-Region replica secrets
- Charged per secret per month

PARAMETER STORE

Built for configuration

- SecureString encrypted with KMS
- No native rotation at all
- Standard tier is free
- Hierarchies and parameter policies

THE TIEBREAKER

Both encrypt with KMS. Rotation and cross-account resource policies are what you are actually paying Secrets Manager for.

Rotation

- ◆ **Managed rotation** for RDS, Aurora, Redshift and DocumentDB. AWS runs it; you set a schedule.
- ◆ Otherwise a **Lambda rotation function**: createSecret, setSecret, testSecret, finishSecret.
- ◆ **Staging labels**: **AWSCURRENT** live, **AWSPENDING** rotating in, **AWSPREVIOUS** the one before.
- ◆ **Single-user** changes one password. **Alternating-user** swaps between two, so one is always valid.
- ◆ Alternating-user needs a superuser secret to clone from. Use it when a failed connection is unacceptable.

ON THE EXAM

Applications should fetch the secret, not cache it. A cached credential outlives its rotation.

Sharing and surviving

- ◆ A **resource policy** grants cross-account access. The other account also needs **kms:Decrypt**.
- ◆ You cannot share a secret encrypted with the **AWS managed key**. The module 10 snapshot rule again.
- ◆ **Replica secrets** copy to other Regions, synced from the primary, each with its own KMS key.
- ◆ Replication is how a secret survives a Regional failure – the module 27 argument, different service.
- ◆ Deletion has a **recovery window of 7 to 30 days**. Force deletion is irreversible.

ON THE EXAM

Nobody can be granted an AWS managed key, so nothing encrypted with one can be shared.

Parameter Store, and where it fits

- ◆ **SecureString** encrypts with KMS. **Standard tier is free** up to 10,000 parameters.
- ◆ **Advanced tier** raises limits and adds **parameter policies** – expiration, and no-change notification.
- ◆ **Hierarchies** grant access by path: a role reaches `/prod/app-a/*` and nothing else.
- ◆ It can **reference a Secrets Manager secret**, so apps read one API.
- ◆ **No rotation.** Automatic rotation in a requirement rules it out.

ON THE EXAM

Configuration in Parameter Store, credentials in Secrets Manager. Use both.

Masking: skill 5.3.4, across six services

- ◆ **CloudWatch Logs data protection** – mask sensitive data in log events, at ingestion.
- ◆ **Bedrock Guardrails sensitive information filters** – block or mask PII in prompts and responses, from module 18.
- ◆ **AWS WAF data protection in web ACLs** – redact fields from WAF logs at source, from module 12.
- ◆ **Redshift dynamic data masking** – the same column returns different values to different roles, with no separate view.
- ◆ **Lake Formation** – column, row and cell-level filters. **S3 Object Lambda** – transform objects on read, so the stored object is untouched.

ON THE EXAM

Macie and Comprehend find the sensitive data. These six do something about it. The exam usually wants the second group.

CloudWatch Logs data protection, precisely

- ◆ **Managed or custom data identifiers.** Detection uses machine learning and pattern matching.
- ◆ Masked at **every egress point** – Insights, metric filters and subscription filters.
- ◆ **logs:Unmask** reveals real values, via the unmask parameter or query command.
- ◆ **Masking happens at ingestion. Earlier events are not masked.** Nothing here is retroactive.
- ◆ Account and log-group policies are **cumulative**. A free **LogEventsWithFindings** metric fires on detection.

ON THE EXAM

Audit mode writes JSON findings to Logs, S3 or Firehose – measure before masking.

The vocabulary, because the exam uses it

- ◆ **Masking** – hidden on display. The original still exists underneath.
- ◆ **Tokenisation** – replaced by a token, mapping held elsewhere. Reversible.
- ◆ **Pseudonymisation** – identifiers replaced consistently so records still join. Reversible, and still personal data.
- ◆ **Anonymisation** – irreversible. If it can be reversed, it is pseudonymisation.
- ◆ The word to listen for is **irreversible**.

ON THE EXAM

Most 'anonymised' datasets are pseudonymised. The distinction is legal as well as technical.

A retailer stores database credentials as SecureString parameters in Parameter Store. A new control requires credentials to rotate every 30 days automatically, to be available in a second Region for disaster recovery, and to be readable by an analytics account.

The application connects continuously and cannot tolerate a failed connection during rotation.

What should the retailer implement?

- A Parameter Store advanced tier with a parameter policy expiring the value every 30 days
- B Secrets Manager with alternating-user rotation, a replica secret, and a resource policy for the analytics account
- C A scheduled Lambda function that rewrites the Parameter Store value every 30 days
- D Parameter Store with a customer managed KMS key shared with the analytics account

◆ PAUSE AND DECIDE

What should the retailer implement?

A Advanced tier with an expiration policy
A parameter policy expires or notifies. It does not generate a new credential or change it in the database – the parameter simply stops being valid.

C A scheduled Lambda rewriting the parameter
Rebuilding Secrets Manager by hand, without staging labels – so there is a window where the stored value and the database disagree.

B Secrets Manager: alternating-user rotation, replica secret, resource policy
Correct. Three requirements, three features: alternating-user avoids a connection gap, replica secrets cover the second Region, resource policy covers the analytics account.

D Parameter Store with a shared customer managed key
Solves cross-account reading and neither of the other two. Parameter Store has no rotation and no replication.

THE SIGNAL

Four requirements, and only one option addresses more than one of them. Count what each option actually covers.

A fintech discovers its payment service has been logging full card numbers into CloudWatch Logs for eleven months. The team immediately applies an account-level data protection policy with the credit card number managed data identifier.

New log events are correctly masked. A spot check of log events from three months ago still shows full card numbers to any user who can read the log group.

Why, and what should the team do about the historical data?

- A Masking applies at ingestion only; historical events must be deleted or their log groups access-restricted
- B The policy must be applied at log group level as well; account-level policies do not cover existing groups
- C The spot check was run by a user with logs:Unmask; remove that permission and the data will appear masked
- D Managed data identifiers take up to 24 hours to apply retroactively; wait and re-check

◆ PAUSE AND DECIDE

Why, and what should the team do about the historical data?

- A

Ingestion only; delete or restrict the historical events

Correct. AWS states it plainly: events ingested before the policy was set are not masked. Nothing in AWS logging is retroactive – the module 1 lesson.
- B

Account-level does not cover existing groups

The opposite. An account-level policy applies to existing log groups and future ones, and the two levels are cumulative.
- C

The checker had logs:Unmask

Worth verifying, and the scenario says any user who can read the group sees the numbers – so it is not a permission held by one person.
- D

Retroactive within 24 hours

Invented. There is no retroactive masking at any delay, which is exactly what makes the eleven months of history a real problem.

THE SIGNAL

Eleven months of history is now the incident. The policy fixed the future, and the past needs deletion or containment.

A support team of 60 engineers needs to query application logs to diagnose customer issues. The logs contain email addresses and phone numbers. Compliance requires that support engineers never see those values, while the four-person incident response team must be able to when investigating.

The team wants one mechanism, applied once, that covers the console, Logs Insights and the subscription filter feeding their SIEM.

What should they implement?

- ☐ A Separate log groups for sensitive and non-sensitive events, with different IAM policies
- ☐ B A Macie job that identifies PII in the logs and alerts the compliance team
- ☐ C A Lambda subscription filter that strips PII before the logs reach the SIEM
- ☐ D A data protection policy masking those identifiers, granting logs:Unmask only to the incident response team

◆ PAUSE AND DECIDE

What should they implement?

A Separate log groups by sensitivity

Requires the application to know which events are sensitive before writing them - which is the assumption that failed in the first place.

C Lambda stripping PII before the SIEM

Covers one egress point of three. Engineers reading the console or running Insights queries would still see everything.

B A Macie job alerting compliance

Macie discovers sensitive data in S3, and it reports rather than redacts. It also tells compliance rather than hiding anything from the 60 engineers.

D Data protection policy, with logs:Unmask only for the IR team

Correct. Masking applies at every egress point - console, Insights and subscription filters - and logs:Unmask is exactly the two-tier access the requirement describes.

THE SIGNAL

'One mechanism covering three egress points' is the phrase. Masking at every egress point is a documented property.

A bank's analysts query a Redshift table containing card numbers. Analysts must see only the last four digits. The fraud team must see full numbers. Both groups query the same tables with the same tools, and the bank does not want to maintain duplicate views or copies of the data.

A junior engineer proposes creating a masked view for analysts and granting the fraud team access to the base table.

What should the bank use instead?

- ☐ A S3 Object Lambda to redact the values as the data is read
- ☐ B Macie automated sensitive data discovery on the Redshift cluster
- ☐ C Redshift dynamic data masking policies attached to the analyst and fraud roles
- ☐ D The junior engineer's proposal, with a Config rule detecting direct base-table grants

◆ PAUSE AND DECIDE

What should the bank use instead?

A S3 Object Lambda

Transforms S3 objects on read, which is genuinely a masking tool. These queries go to Redshift tables, and Object Lambda sits in the S3 GET path.

C Redshift dynamic data masking per role

Correct. The same column returns different values depending on the querying role, with no duplicate views and no second copy of the data.

B Macie on the Redshift cluster

Macie discovers sensitive data in S3. It reports what is there rather than changing what a query returns – the module 4 discipline again.

D Masked view plus a Config rule

Two objects to keep in step, and the Config rule detects a wrong grant after it has been made. The requirement explicitly rules out duplicate views.

THE SIGNAL

'Same tables, same tools, no duplicates' describes dynamic masking exactly. A view is a second object that can drift.

Six things to carry forward

- ◆ **Parameter Store has no rotation.** Automatic rotation means Secrets Manager.
- ◆ **Alternating-user rotation** when a failed connection is unacceptable.
- ◆ **You cannot share anything encrypted with an AWS managed key.**
- ◆ **Log masking applies at ingestion.** Historical events are not masked.
- ◆ **Masking covers every egress point;** logs:Unmask is the exception.
- ◆ **Irreversible means anonymisation.** The other three are reversible by someone.

ON THE EXAM

That closes Domain 5. Next: Domain 6, governance – Organizations and the control hierarchy.